2024

WORD COUNT 4305
TIME SUBMITTED 26-NOV-2024 10:02PM
PAPER ID 113170200

Landmark Selection Strategy to Accelerate Shortest Distance Queries

Zhipeng He
Guangzhou University
Guangzhou, China
2112233062@e.gzhu.cn

Dian Ouyang*
Guangzhou University
Guangzhou, China
dian.ouyang@gzhu.edu.cn

Jianye Yang
Guangzhou University
Guangzhou, China
jyyang@gzhu.edu.cn

***Abstract*—Computing the shortest path between vertices is a fundamental task in network analysis. This paper introduces an innovative strategy for selecting landmarks in a graph, which improves upon the traditional highest-degree selection method. Our approach captures the essential structural features of graphs, effectively reducing landmark overlap and clustering. Combined with partition querying and upper-bound pruning optimizations, our algorithm achieves a notable $20\% - 30\%$ increase in query speed and is applicable to multi-million record datasets. This study offers new insights and solutions for enhancing query performance in large-scale graph-structured data, paving the way for further innovations in graph database optimization to meet the growing demands of large-scale graph data processing.**

***Keywords*-Shortest path, landmark, distance labeling, optimization**

I. Introduction

In dealing with large-scale graph data, finding the shortest path is a core issue in data mining and network analysis, with a wide range of applications including, but not limited to, traffic planning, social network analysis, and network routing optimization [2], [5], [11], [18]. As the scale of data continues to grow, traditional shortest path algorithms face significant challenges in terms of efficiency and scalability [15], [18]. How to quickly and accurately calculate the shortest path has become an urgent problem to solve.

Current state-of-the-art algorithms in shortest path computations leverage landmark labeling-based approaches, which typically require specifying a parameter 'K' to denote the number of landmarks [2], [6], [7]. These algorithms conventionally select the top 'K' vertices with the highest degree values within the graph to serve as landmarks.

A. Motivation

We believe that using only the top k vertices with the highest degree as the landmark may have optimization space. To try different strategies, we considered using the top k vertices with the highest betweenness centrality as the landmark. However, although there has been some improvement in label size, it has not shown significant improvement in actual queries [3]. More importantly, due to the high time-consuming burden of calculating betweenness centrality in large graphs, we have to abandon this method. By analyzing the strategy of directly using vertices with the highest degree and betweenness centrality as landmarks [2], [3], [6]–[9], we found two main issues: firstly, the landmarks selected based on betweenness centrality overlap with those selected by the degree, and this situation is particularly prominent in large networks. Secondly, in highly clustered networks, strong clustering may lead to a core–periphery structure. [4], [17]. The selected vertices are relatively clustered, and we hope they can be dispersed to better cover the entire graph. Therefore, we are trying to find new algorithmic inspirations.

B. Our idea

We propose a novel landmark selection strategy: (1) Firstly, we use the Top-k degree vertices as benchmark landmarks and assign other vertices to the region where their nearest landmarks are located. (2) Next, we further select the vertex with the highest degree in each region to serve as the new landmark. This process aims to solve the problems of vertex overlap and clustering, improving query efficiency. It is important to emphasize that in our new landmark selection, we only focus on the highest degree vertices within each region, without considering vertices across regions. This detail ensures that the new landmark is scattered throughout the entire graph, rather than clustered in a specific local area. (3) In the query phase, we further introduced optimization of partitioned queries.

C. Contribution

In this paper, our main contributions are as follows:

- An innovative strategy for selecting landmarks in a graph has been proposed, which better captures the important features of the graph structure compared to traditional highest degree selection methods, effectively reducing the overlap and clustering of landmarks.
- The concept of region partitioning has been introduced, which divides vertices into the region where the nearest landmark is located, making the landmark more dispersed throughout the graph, reducing its clustering and improving query efficiency.
- The proposed landmark selection strategy and query optimization method have good scalability and universality, and are suitable for various practical application scenarios such as graph databases and social networks. They still perform well in processing billion level

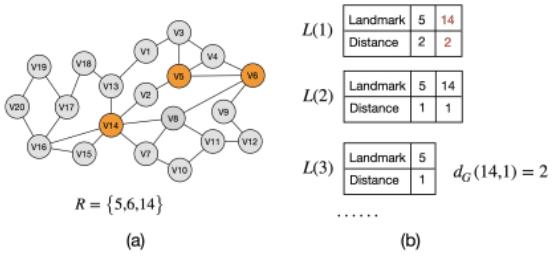

Figure 1. An explanation of distance labeling:(a) an example graph G, (b) Compute a label $L(v)$ for each vertex $v \in V$ in G in advance.

datasets, providing strong support for research and application in related fields.

D. Outline

The paper is structured as follows. In Section II, we introduce the problem definition and preliminary. We briefly describe the existing solutions in Section III. In Section IV, we introduce the Distance Query Framework, while Section V is dedicated to optimizing the landmark selection strategy. Section VI presents experiments that evaluate our methods. Finally, Section VII concludes this paper.

II. Preliminary

Let $G = (V, E)$ be a graph, where V is the set of vertices and E is the set of edges. We denote the number of vertices and edges by $n = |V|$ and $m = |E|$, respectively. For simplicity, we assume that the G is connected and undirected throughout this paper, since our work can be easily extended to directed or disconnected graphs. Let $G[V \setminus R]$ represent removing landmarks set in R from G, and $d_G(s,t)$ represent the shortest distance from s to t.

A. Distance labeling

Let $R \subseteq V$ be a set of special vertices in G, called landmarks, a label $L(v)$ for each vertex $v \in V$ can be precomputed, which can be answered in linear-time by looking up the label $L(v)$. The collection of labels $L = \{L(v)\}_{v \in V}$ is referred to as a distance labeling on graph G. The distance between two vertices s and t can be calculated as:

$$\mathrm{d}_G(s,t) = \min_{r \in L(s) \cap L(t)} (\mathrm{d}_G(s,r) + \mathrm{d}_G(r,t))$$

Example 1. *Consider the graph G depicted in Fig.1(a), with a set of landmarks $R = \{5, 6, 14\}$ and precomputed distance labels $L(1) = \{(5,2),(14,2)\}$, $L(2) = \{(5,1),(14,1)\}$ and $L(3) = \{(5,1)\}$, etc. in Fig.1(b), we can efficiently determine that $d_G(14,1) = 2$.*

B. Problem Definition

Given a graph G and a set of precomputed landmarks $R \subseteq V$, a shortest distance query $q = (s,t)$ is defined, where $s, t \in V$, and the answer is the shortest distance $d_G(s,t)$. This paper aims to develop efficient landmark selection strategies to optimize query performance. Specifically, our objective is to minimize the total query time $T(Q) = min \sum_{q \in Q} T(q,R)$ where $T(q,R)$ represents the time taken to answer the query q using the set of landmarks R.

Example 2. *Fig.1(a) shows a network $G = (V,E)$ with 20 vertices and 30 edges. There are many paths between v_{12} and v_{19}. For example, two of them are $p_1 = (v_{12}, v_{11}, v_8, v_{14}, v_{13}, v_{18}, v_{17}, v_{19})$ and $p_2 = (v_{12}, v_9, v_6, v_5, v_2, v_{14}, v_{16}, v_{20}, v_{19})$ with $d_G(p_1) = 7$ and $d_G(p_2) = 8$. p_1 is a shortest path between v_{12} and v_{19} as covered by landmark v_{14}. Given a shortest distance query $q = (v_{12}, v_{19})$, the answer of q is $d(v_{12}, v_{19}) = 7$.*

III. Existing Solutions

This section formalizes two distance labeling concepts: 2-hop labeling and Highway cover labeling [1], [7], which lay the foundation for our theoretical analysis and new algorithm.

A. 2-Hop Labeling

The 2-hop labeling technique is commonly used in methods for shortest path computation [2], [6], [7], [11]–[13], [15], [16], [18]. Given a network G, 2-hop labeling assigns each vertex $v \in V$ a label $L(v)$ which is a collection of pairs $(u, \mathrm{d}_G(v,u))$ where $u \in V$.

Definition 1. ***(2-hop cover labeling).*** *A labeling L is a 2-hop Cover labeling of G if, for each pair $s, t \in V$, $L(s) \cap L(t)$ contains at least one common vertex r on the shortest path from s to t. This guarantees that:*

$$Query(s,t,L) = \min\{\delta_L(r,s) + \delta_L(r,t) \mid r \in L(s) \cap L(t)\}$$

Thus, we determine $d_G(s,t)$ by scanning both $L(s)$ and $L(t)$, with a query time complexity of $O(|L(s)| + |L(t)|)$.

B. Highway Cover Labeling

We start by providing the definitions of highway and highway cover labeling scheme [7], [10].

Definition 2. ***(Highway).*** *A highway H can be defined as a tuple (R, δ_H) consists of a distance decoding function δ_H and a set R of landmarks, the value δ_H corresponds to the shortest path distance $d_G(r_1, r_2)$, i.e. $\delta_H(r_1, r_2) = d_G(r_1, r_2)$.*

Example 3. *Refer to the graph G shown in the Fig. 2, we select the set $R = \{5, 8, 14\}$ as landmarks. The distance decoding function δ_H is specified as follows: $\delta_H(5, 14)$ denotes the shortest path distance from vertices 5 to 14, which is 2. Similarly, $\delta_H(5, 8)$ is 2 and $\delta_H(8, 14)$ is 1.*

This highway H effectively represents the landmark distance relationships within graph G, facilitating the rapid computation or estimation of shortest paths between vertices.

Figure 2. Highway Labeling

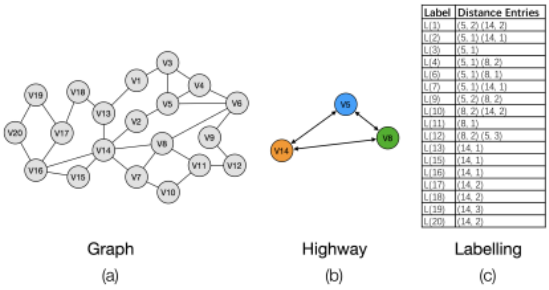

Figure 3. Highway Cover Labeling

Definition 3. ***(Highway Cover Labeling)***. *A highway cover labelling* $\Gamma = (H, L)$ *consists of a highway* H *and a distance labeling* L. *Then for any vertices* $u \in V \backslash R$ *and for any* $r \in R$, *there exist* $r' \in R$ *in* $L(u)$ *such that* r' *lies on the shortest path from* u *to* r *(where* r *and* r' *may be the same).*

Example 4. *In the network illustrated in the Fig. 3(a), the highway* H *includes three landmarks:* 5, 8, *and* 14, *highlighted in Fig. 3(b). The path* $p_1 = (v_3, v_5, v_4)$ *denotes the shortest path between vertices* 3 *and* 4, *which is constrained by landmark* 5. *In contrast, neither of paths* $p_2 = (v_3, v_1, v_{13}, v_{14}, v_2, v_5, v_4)$ *and* $p_3 = (v_3, v_4)$ *satisfies the condition of being a shortest path between 3 and 4 constrained by landmark 5.*

In contrast to a 2-hop cover labeling, a highway cover labeling can only answer distance queries between landmarks and other vertices in the graph, making it a partial distance labeling [12].

How to choose a better landmark set will be explained in detail in the section V, which is the main part of our algorithm.

IV. Distance Query Framework

In this section, we present the distance query framework, which facilitates efficient and accurate batch computation of shortest distances for any pair of vertices in a large graph.

A. Label Construction

The label construction in our algorithm is outlined in Algorithm 1. The procedure begins by performing a Breadth-

Algorithm 1: Construct highway cover labeling L

Input: Network $G(V, E)$, Highway $H(R, \delta_H)$
Output: The highway cover labeling L

$L(v) \leftarrow \emptyset$ for all $v \in V \setminus R$;
foreach $r_i \in R$ **do**
  $Q_L \leftarrow \emptyset$;
  $Q_N \leftarrow \emptyset$;
  Q_L.push(r_i);
  $n \leftarrow 0$;
  while Q_L *and* Q_N *are not empty* **do**
    foreach $u \in Q_L$ *at depth* n **do**
      foreach *unvisited neighbor* v *of* u **do**
        $\delta_{\text{BFS}}(r_i, v) \leftarrow n + 1$;
        if v *is a landmark* **then**
          Q_N.push(v);
        else
          Q_L.push(v);
          $L(v) \leftarrow L(v) \cup \{(r_i, \delta_{\text{BFS}}(r_i, v))\}$;
    foreach $u \in Q_N$ *at depth* n **do**
      foreach *unvisited neighbor* v *of* u **do**
        $\delta_{\text{BFS}}(r_i, v) \leftarrow n + 1$;
        Q_N.push(v);
    $n \leftarrow n + 1$;
return L

First Search(BFS) from each landmark $r \in R$. During this process, a distance entry $(r, \delta(r, v))$ is added to the label of a vertex $v \in V \setminus R$ if no closer landmark $r' \in R$ lies on the shortest path from r to v. This condition ensures that the label for v reflects the shortest distance from the most proximal landmark, optimizing the search process in our graph queries.

In Fig. 4, the green vertices are visited and stored, while the grey vertices are pruned and no stored. This is because when BFS traversal begins at landmark r, grey vertices are obstructed by other landmarks r'.

B. Query Distance

For any pair of vertices s and t, a highway cover distance labeling scheme L can be used to estimate an upper bound $\underline{d}_{s,t}$ for the shortest path distance from s to t can be determined as follows,

$$\underline{d}_{s,t} = distance[s][i] + distance[t][i] \quad (1)$$

$$\underline{d}_{s,t} = distance[s][i] + highway[i][j] + distance[t][j] \quad (2)$$

Here, Equation (1) represents the shortest distance from s to t passing through the landmark r_i. Equation (2) represents the shortest distance from s to t passing through both

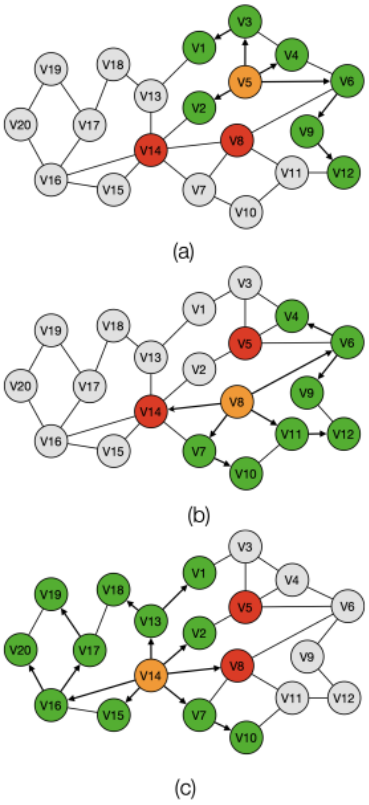

Figure 4. Label Construction.

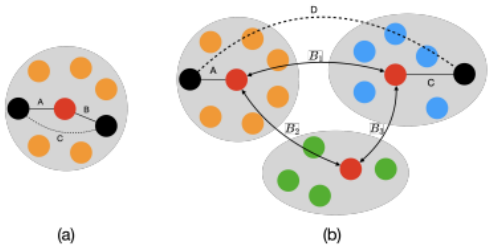

Figure 5. Partition Query

landmarks r_i and r_j, where the distance between r_i and r_j can be directly obtained from the highway.

We conduct distance query as described in Algorithm 2. When s and t are in the same region, the upper bound is calculated using Equation (1). Conversely, when s and t are in different regions, the upper bound is computed using Equation (2). The next section we will provide a detailed explanation of how the regions are partitioned.

Next, we introduce hash region and partition query for further optimization. In our algorithm, hash region technique is a kind of hash table to quickly determine if vertices s and t belong to the same region. During the query process, we first Check if s and t are in the same region using the hash region. If they are, distance[s][i] + distance[t][i] is used as the upper bound pruning condition. If not, distance[s][i] + highway[i][j] + distance[t][j] serves as the upper bound. Subsequently, we conduct a simultaneous bidirectional search initiated from both s and t in $G[V \setminus R]$ under the upper bound $\underline{d}_{s,t}$.

Algorithm 2: Query Distance

Input: s, t
Output: $d_G(s,t)$
if $vertexToRegion[s] == vertexToRegion[t]$ **then**
  Initialize queues and distances;
  $dist\_upper \leftarrow$
   $landmark\_distances[s][region] +$
   $landmark\_distances[t][region]$;
  while *both queues are not empty* **do**
    Select the smaller queue;
    while *selected queue is not empty* **do**
      foreach w *in neighbors of current node* **do**
        if w *already visited by other queue* **then**
          return path length via w;
        Mark w and push to queue;
  Reset and clear queues;
  return $\min(res, dist\_upper)$;
else
  Initialize $m \leftarrow 99$;
  foreach i *in* $C[s]$ **do**
    foreach j *in* $C[t]$ **do**
      $m \leftarrow \min(m, distances[s][i] +$
       $highway[vertices[s][i]][vertices[t][j]] +$
       $distances[t][j])$;
  return m;

Example 5. *Refer to the graph shown in Fig. 5, black vertices represent the vertices to be queried, and red vertices represent the finalized landmarks. In Fig. 5(a), after region-based hash determines that s and t belong to the same region, a bidirectional BFS is executed. The process terminates when the distance C exceeds $A + B$, at which $d_G(s,t) = A + B$ is confirmed as the shortest distance. In Fig. 5(b), orange, green, and blue vertices each represent vertices within the same region. After region-based hash determines that s and t do not belong to the same region, a bidirectional BFS is executed. The process terminates once the distance D exceeds $A + \min\{B1, B2 + B3\} + C$, establishing $d_G(s,t) = A + \min\{B1, B2 + B3\} + C$ as the*

shortest distance. Note that, B_1, B_2 and B_3 can be directly obtained through the highway.

V. Landmark Selection

In this section, we propose an innovative strategy for selecting important vertices, referred to as landmarks, in a graph. This approach seeks to improve upon the traditional method of selecting the top-K vertices with the highest degrees by incorporating a more nuanced iterative regional division process.

Algorithm 3: Landmark Selection

Input: $G = (V, E)$;
Output: $topk[]$;
Function `SelectLandmarks` ($Graph\ G$)**:**
  $topk[] \leftarrow$ select K highest degree vertices as landmarks;
  foreach *landmark* j *in* $topk$ **do**
    Perform BFS from j to calculate distances;
  end
  foreach $i = 0$ **to** $V - 1$ **do**
    if i *not in* $topk$ **then**
      Find the nearest landmark for vertex i and assign to its region
    end
  end
  foreach $i = 0$ **to** $K - 1$ **do**
    foreach *vertex in* $regions[i]$ **do**
      Find MaxDgreeVertex;
    end
    $topK[vertex] \leftarrow MaxDgreeVertex$
  end
  return $topk[]$;

Algorithm 3 describes how our algorithm select landmarks. The strategy enhances efficiency and effectiveness in graph processing tasks. The detailed methodology is follows: (i) **Initial Step:** We begin by selecting the top-K vertices with the highest degrees in the graph as the initial set of landmarks. This step serves as the foundation for the subsequent iterative process, providing initial vertices for the refinement of landmark selection. (ii) **Regional Division Step:** Next, each vertex not already designated as a landmark is assigned to the region associated with the nearest landmark. This division creates a dedicated influence zone for each landmark, simplifying the search space for subsequent queries and thus enhancing query speed. (iii) **New Landmark Selection Step**: Each region is considered independently, which only considers the connections within the region and ignores the connections across regions. On this basis, we select a vertex with the highest degree within each region as a new landmark to ensure efficiency and accuracy in query operations.

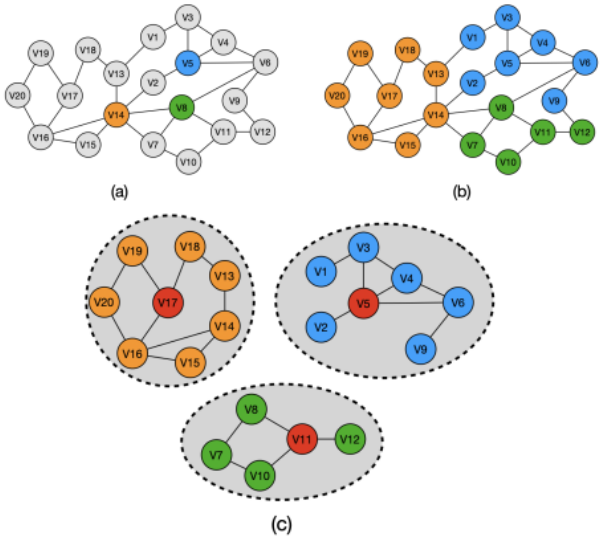

Figure 6. Landmark Selection.

Through such strategy, we construct a set of landmarks that not only considers vertex significance but also optimizes spatial locality, significantly speeding up subsequent algorithm queries. The essence of this method lies in its ability to rapidly locate the area relevant to a query, reducing unnecessary search and computation, offering a new pathway for efficient processing of large-scale graph data.

It should be noted that due to the Order Independence [7] property of highway cover labeling, there is no need to consider the attribution of a vertex to different landmarks at the same distance. The vertices that are first traversed belong to the region covered by that landmark.

Proof: Defind $OrderL_1$ and $OrderL_2$ as two distance labeling orders over landmark R. During the label construction process, the BFS conducted from landmark r in $OrderL_1$ and $OrderL_2$ traverses the same set of vertices, resulting the same pruned BFS tree. Thus, Algorithm 1 ensures that $L_1(v) = L_2(v)$ for every $v \in V \backslash R$. ■

Example 6. *In the network illustrated in Fig. 6(a), vertices $\{v_5, v_8, v_{14}\}$, which have the highest degrees, are selected as initial landmarks. Starting from each landmark, recording the vertices reachable by each landmark, as shown in Fig. 6(b). Subsequently, as detailed in Fig. 6(c), each region is treated independently, with marginalization of vertices, meaning only intra-region edges are counted while inter-region connections are excluded. Within each region, the vertex with the highest degree, vertices $\{v_5, v_{11}, v_{17}\}$ marked in red, is selected as the final landmark.*

VI. Experiment

In this section, I present extensive experiments to evaluate our methods. All algorithms were implemented in C++11 using the Standard Template Library (STL) and tested on a server with an Intel Xeon 2.10GHz CPU, 256GB of RAM, and Ubuntu 20.04. The experimental code was written in C++.

A. Datasets

Table I
Datasets

Network	Vertices n	Edges m
Douban	154,909	654,324
Amazon	334,863	1,851,744
DBLP	317,980	2,099,732
Youtube	1,134,890	5,975,248
As-skitter	1,696,415	22,190,596

We use five publicly available real-world networks from the Stanford Network Analysis Project [14]. Table I provides details of these datasets, which we treated as undirected graphs.

B. Query Performance Comparison

Table II
Random vertex pair shortest distance batch query

Datasets	HL	Naive-KHL	KHL
douban	0.806s	0.799s	0.613s
amazon	5.863s	5.841s	4.201s
dblp	1.96s	1.791s	1.538s
youtube	6.584s	5.863s	5.164s
as-skitter	11.816s	11.174s	8.933s

To evaluate the average query time, we randomly generate one million vertex pairs from each graph for batch processing of shortest path distance queries. The results are shown in Table II. Our algorithm KHL denotes a region hash optimization based on the use of a new landmark selection strategy. The Naive-KHL method denotes no region hash optimization. This optimization is described in Section IV(B). The HL algorithm used in our experiments is derived from [7] which selects the top k highest degree vertices as landmarks without any optimization during the query phase. Experimental validation demonstrated that our algorithm enhances query speed by 20%-30% in large networks compared to traditional methods.

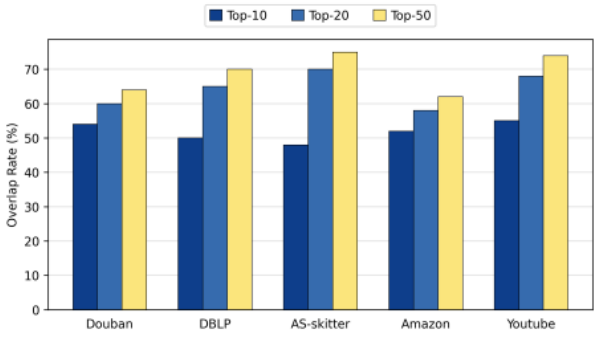

Figure 7. Overlap rates

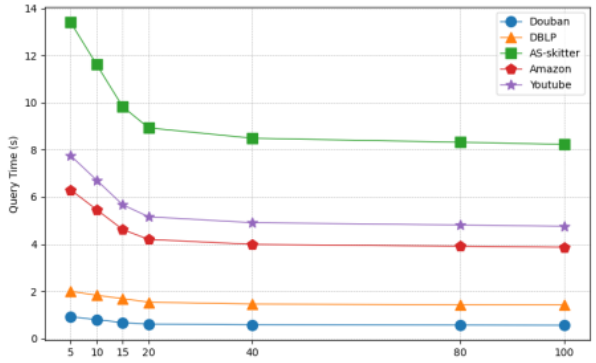

Figure 8. Performance under Varying Landmarks

C. Overlap rates

We conducted experimental analyses on vertices selected as landmarks using both the Top-k degree and Top-k betweenness methods. The horizontal axis represents the datasets, and the vertical axis denotes the overlap rate. We compared the overlap ratio of vertices selected as landmarks under conditions where the number of landmarks was set to 10, 20, and 50, respectively. Fig. 7 shows that these two traditional methods have a high overlap rates in selected landmarks. Due to the properties of degree, vertices near those with a higher degree also tend to have higher degrees, resulting in a clustered arrangement of landmarks. The experiments indicated that the high overlap rate also leads to clustering when landmarks are selected based on betweenness. Thus, ensuring a dispersed distribution of landmarks, which can cover more shortest paths between vertex pairs, is a crucial optimization. Our method ensures zero overlap with the landmarks selected by traditional methods.

D. Performance under Varying Landmarks

From Fig. 8, We can see that: As the number of landmarks increases, the query speed also improves. However, selecting

additional landmarks results in increased preprocessing time and storage space. A diminishing return is observed when the count exceeds 20 landmarks, making further additions ineffective. Therefore, we consider 20 landmarks to be an optimal choice.

VII. Conclusion

In this study, we proposed and implemented an innovative Landmark selection strategy that improves upon the traditional degree-based approach, significantly enhancing the selection efficacy of Landmarks while reducing redundancy and clustering. By integrating partition querying and upper-bound pruning optimizations, our algorithm achieved a notable improvement of 20% to 30% in query speed and can be applied to datasets comprising tens of millions of records. This research provides new insights and 42 tions for enhancing query performance in large-scale graph-structured data, such as graph databases and social networks. Moving forward, we will continue to explore more innovative strategies for optimizing graph databases to meet the growing demands of large-scale graph data processing.

References

[1] Cohen, E., Halperin, E., Kaplan, H., Zwick, U.: Reachability and distance queries via 2-hop labels. SIAM Journal on Computing **32**(5), 1338–1355 (2003)

[2] Wang, Y., Wang, Q., Koehler, H., Lin, Y.: Query-by-sketch: Scaling shortest path graph queries on very large networks. In: Proceedings of the 2021 International Conference on Management of Data, pp. 1946–1958 (2021)

[3] Zhang, Q., Li, R.-H., Pan, M., Dai, Y., Wang, G., Yuan, Y.: Efficient top-k ego-betweenness search. In: 2022 IEEE 38th International Conference on Data Engineering (ICDE), pp. 380–392 (2022)

[4] W. Li, M. Qiao, L. Qin, Y. Zhang, L. Chang, and X. Lin, "Scaling up distance labeling on graphs with core-periphery properties," in *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data*, 2020, pp. 1367–1381.

[5] D. Ouyang, L. Yuan, L. Qin, L. Chang, Y. Zhang, and X. Lin, "Efficient shortest path index maintenance on dynamic road networks with theoretical guarantees," *Proceedings of the VLDB Endowment*, vol. 13, no. 5, pp. 602–615, 2020.

[6] Akiba, T., Hayashi, T., Nori, N., Iwata, Y., Yoshida, Y.: Efficient top-k shortest-path distance queries on large networks by pruned landmark labeling. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 29, no. 1 (2015)

[7] Farhan, M., Wang, Q., Lin, Y., Mckay, B.: A highly scalable labelling approach for exact distance queries in complex networks. arXiv preprint arXiv:1812.02363 (2018)

[8] D. He, P. Yuan, and H. Jin, "Answering reachability queries with ordered label constraints over labeled graphs," *Frontiers of Computer Science*, vol. 18, no. 1, p. 181601, 2024.

[9] J. Zhang, W. Li, L. Yuan, L. Qin, Y. Zhang, and L. Chang, "Shortest-path queries on complex networks: experiments, analyses, and improvement," *Proceedings of the VLDB Endowment*, vol. 15, no. 11, pp. 2640–2652, 2022.

[10] Akiba, T., Iwata, Y., Kawarabayashi, K.-i., Kawata, Y.: Fast shortest-path distance queries on road networks by pruned highway labeling. In: Proceedings of the Sixteenth Workshop on Algorithm Engineering and Experiments (ALENEX), pp. 147–154 (2014)

[11] Ouyang, D., Qin, L., Chang, L., Lin, X., Zhang, Y., Zhu, Q.: When hierarchy meets 2-hop-labeling: Efficient shortest distance queries on road networks. In: Proceedings of the 2018 International Conference on Management of Data, pp. 709–724 (2018)

[12] M. Farhan, Q. Wang, and H. Koehler, "BatchHL: Answering distance queries on batch-dynamic networks at scale," in *Proceedings of the 2022 International Conference on Management of Data*, pp. 2020–2033, 2022.

[13] Zheng, B., Wan, J., Gao, Y., Ma, Y., Huang, K., Zhou, X., Jensen, C.S.: Workload-aware shortest path distance querying in road networks. In: 2022 IEEE 38th International Conference on Data Engineering (ICDE), pp. 2372–2384 (2022)

[14] SNAP Datasets. Available at http://snap.stanford.edu/data/index.html.

[15] Abraham, I., Delling, D., Goldberg, A.V., Werneck, R.F.: Hierarchical hub labelings for shortest paths. In: European Symposium on Algorithms, pp. 24–35 (2012)

[16] Abraham, I., Delling, D., Goldberg, A.V., Werneck, R.F.: A hub-based labeling algorithm for shortest paths in road networks. In: Experimental Algorithms: 10th International Symposium, SEA 2011, Kolimpari, Chania, Crete, Greece, May 5-7, 2011. Proceedings 10, pp. 230–241 (2011)

[17] Wikipedia contributors. Clustering Coefficient. Wikipedia, The Free Encyclopedia. Available at https://en.wikipedia.org/wiki/Clustering_coefficient.

[18] Y. Zeng, K. Li, X. Zhou, W. Luo, and Y. Gao, "An efficient index-based approach to distributed set reachability on small-world graphs," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 10, pp. 2358–2371, 2022.

2024

ORIGINALITY REPORT

21%
SIMILARITY INDEX

PRIMARY SOURCES

1	Wang, Ye. "Exploring Shortest Paths on Large-Scale Networks", The Australian National University (Australia), 2022 ProQuest	94 words — 2%
2	openresearch-repository.anu.edu.au Internet	79 words — 2%
3	Dian Ouyang, Lu Qin, Lijun Chang, Xuemin Lin, Ying Zhang, Qing Zhu. "When Hierarchy Meets 2-Hop-Labeling", Proceedings of the 2018 International Conference on Management of Data - SIGMOD '18, 2018 Crossref	46 words — 1%
4	Bolong Zheng, Jingyi Wan, Yongyong Gao, Yong Ma, Kai Huang, Xiaofang Zhou, Christian S. Jensen. "Workload-Aware Shortest Path Distance Querying in Road Networks", 2022 IEEE 38th International Conference on Data Engineering (ICDE), 2022 Crossref	36 words — 1%
5	Dian Ouyang, Dong Wen, Lu Qin, Lijun Chang, Xuemin Lin, Ying Zhang. "When hierarchy meets 2-hop-labeling: efficient shortest distance and path queries on road networks", The VLDB Journal, 2023 Crossref	36 words — 1%
6	www.coursehero.com Internet	31 words — 1%

7	Muhammad Farhan, Henning Koehler, Qing Wang. "BatchHL$$^{+}$$: batch dynamic labelling for distance queries on large-scale networks", The VLDB Journal, 2023 Crossref	25 words — 1%
8	assets.researchsquare.com Internet	25 words — 1%
9	vdoc.pub Internet	25 words — 1%
10	Lei Zou, Lei Chen, M. Tamer Özsu, Dongyan Zhao. "Answering pattern match queries in large graph databases via graph embedding", The VLDB Journal, 2011 Crossref	17 words — < 1%
11	Lijun Chang, Jeffrey Xu Yu, Lu Qin, Hong Cheng, Miao Qiao. "The exact distance to destination in undirected world", The VLDB Journal, 2012 Crossref	17 words — < 1%
12	link.springer.com Internet	17 words — < 1%
13	"Green, Smart and Connected Transportation Systems", Springer Science and Business Media LLC, 2020 Crossref	15 words — < 1%
14	"Progress in Pattern Recognition, Image Analysis, Computer Vision, and Applications", Springer Nature America, Inc, 2014 Crossref	15 words — < 1%
15	export.arxiv.org Internet	15 words — < 1%

16	"Database Systems for Advanced Applications", Springer Science and Business Media LLC, 2023 Crossref	13 words — < 1%
17	Farhan, Muhammad. "Answering Shortest Path Distance Queries in Large Complex Networks", The Australian National University (Australia), 2022 ProQuest	13 words — < 1%
18	Fang Wei. "Chapter 13 Efficient Graph Reachability Query Answering Using Tree Decomposition", Springer Science and Business Media LLC, 2010 Crossref	12 words — < 1%
19	Ye Wang, Qing Wang, Henning Koehler, Yu Lin. "Query-by-Sketch: Scaling Shortest Path Graph Queries on Very Large Networks", Proceedings of the 2021 International Conference on Management of Data, 2021 Crossref	12 words — < 1%
20	Hongxuan Huang, Qingyuan Linghu, Fan Zhang, Dian Ouyang, Shiyu Yang. "Truss Decomposition on Multilayer Graphs", 2021 IEEE International Conference on Big Data (Big Data), 2021 Crossref	11 words — < 1%
21	mdpi-res.com Internet	11 words — < 1%
22	"Lower Bounds in the Preprocessing and Query Phases of Routing Algorithms", Lecture Notes in Computer Science, 2015. Crossref	10 words — < 1%
23	Da Yan, James Cheng, M. Tamer Özsu, Fan Yang, Yi Lu, John C. S. Lui, Qizhen Zhang, Wilfred Ng. "A	10 words — < 1%

general-purpose query-centric framework for querying big graphs", Proceedings of the VLDB Endowment, 2016
Crossref

24 Waqas Nawaz, Kifayat-Ullah Khan, Young-Koo Lee. "SPORE: shortest path overlapped regions and confined traversals towards graph clustering", Applied Intelligence, 2015 — 10 words — < 1%
Crossref

25 www.vldb.org — 10 words — < 1%
Internet

26 "Web Information Systems and Applications", Springer Science and Business Media LLC, 2024 — 9 words — < 1%
Crossref

27 Dan He, Sibo Wang, Xiaofang Zhou, Reynold Cheng. "GLAD: A Grid and Labeling Framework with Scheduling for Conflict-Aware kNN Queries", IEEE Transactions on Knowledge and Data Engineering, 2019 — 9 words — < 1%
Crossref

28 Giuseppe Sirigu, Mario Cassaro, Manuela Battipede, Piero Gili. "Autonomous taxi operations: algorithms for the solution of the routing problem", 2018 AIAA Information Systems-AIAA Infotech @ Aerospace, 2018 — 9 words — < 1%
Crossref

29 Michael Fire, Carlos Guestrin. "The rise and fall of network stars: Analyzing 2.5 million graphs to reveal how high-degree vertices emerge over time", Information Processing & Management, 2020 — 9 words — < 1%
Crossref

30 Mostafa Haghir Chehreghani, Albert Bifet, Talel Abdessalem. " Adaptive Algorithms for Estimating Betweenness and -path Centralities ", Proceedings of the 28th — 9 words — < 1%

ACM International Conference on Information and Knowledge Management - CIKM '19, 2019
Crossref

31 Yuanyuan Zeng, Wangdong Yang, Xu Zhou, Guoqing Xiao, Yunjun Gao, Kenli Li. "Distributed Set Label-Constrained Reachability Queries over Billion-Scale Graphs", 2022 IEEE 38th International Conference on Data Engineering (ICDE), 2022 — 9 words — < 1%
Crossref

32 dokumen.pub — 9 words — < 1%
Internet

33 Houyu Di, Junhu Wang. "Chapter 23 An Experimental Evaluation ofTwo Methods onShortest Distance Queries overSmall-World Graphs", Springer Science and Business Media LLC, 2024 — 8 words — < 1%
Crossref

34 Jianye Yang, Yun Peng, Wenjie Zhang. "(p,q)-biclique counting and enumeration for large sparse bipartite graphs", Proceedings of the VLDB Endowment, 2021 — 8 words — < 1%
Crossref

35 Mengxuan Zhang, Lei Li, Wen Hua, Xiaofang Zhou. "Efficient 2-Hop Labeling Maintenance in Dynamic Small-World Networks", 2021 IEEE 37th International Conference on Data Engineering (ICDE), 2021 — 8 words — < 1%
Crossref

36 Sibo Wang, Wenqing Lin, Yi Yang, Xiaokui Xiao, Shuigeng Zhou. "Efficient Route Planning on Public Transportation Networks", Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data, 2015 — 8 words — < 1%
Crossref

37	arxiv.org Internet	8 words — < 1%
38	ipfs.io Internet	8 words — < 1%
39	www.cs.newpaltz.edu Internet	8 words — < 1%
40	"Databases Theory and Applications", Springer Science and Business Media LLC, 2020 Crossref	7 words — < 1%
41	"Web Information Systems and Applications", Springer Science and Business Media LLC, 2023 Crossref	7 words — < 1%
42	"Web and Big Data", Springer Science and Business Media LLC, 2024 Crossref	7 words — < 1%
43	Lecture Notes in Computer Science, 2014. Crossref	7 words — < 1%
44	Johannes Blum, Sabine Storandt. "Chapter 31 Customizable Hub Labeling: Properties andAlgorithms", Springer Science and Business Media LLC, 2022 Crossref	6 words — < 1%
45	Lecture Notes in Computer Science, 2012. Crossref	6 words — < 1%
46	Li, Wentao. "Measuring Graphs with Shortest Distances", University of Technology Sydney (Australia), 2023 ProQuest	6 words — < 1%

47 Qiu, Yuxuan. "Efficient Substructure Mining in Large Networks", University of Technology Sydney (Australia), 2023
ProQuest
6 words — < 1%

48 "Machine Learning and Knowledge Discovery in Databases", Springer Science and Business Media LLC, 2014
Crossref
5 words — < 1%

EXCLUDE QUOTES	ON	EXCLUDE SOURCES	OFF
EXCLUDE BIBLIOGRAPHY	ON	EXCLUDE MATCHES	OFF